1º DAM

Programación

Jesús Castellano González

Trimestre 1

Programación Estructurada

Instrucciones de Control de Flujo del Programa (C#)

Unidad Didáctica 2

1.- Instrucciones de Control (C#)

Instrucciones de Control: Decisión Simple

Los programas deben realizar diferentes acciones en respuesta a distintas condiciones.

La instrucción **if** permite evaluar una condición y ejecutar las diferentes secciones de código basándose en los resultados de esa condición.

Si la condición se cumple (**true**) ejecutará el código contenido dentro de la instrucción **if**.

Sintaxis:

```
if (<condición>) {
    <Instrucciones>
}
```

Instrucciones de Control: Decisión Simple

Ejemplo:

```
float ventas;
float descuento;
…
if (ventas > 10000) {
        descuento = 0.10 * ventas;
}
…
```

Instrucciones de Control: Decisión Doble

La cláusula **else** permite especificar un bloque de código que se ejecutará si la condición es **false**.

Sintaxis:

```
if (<condición>) {
    <InstruccionesTrue>
}
else {
    <InstruccionesFalse>
}
```

Instrucciones de Control: Decisión Doble

Ejemplo:

```
float ventas;
float descuento;
…
if (ventas > 10000) {
        descuento = 0.10 * ventas;
}
else {
        descuento = 0;
}
…
```

Instrucciones de Control: Decisión (Anidación)

Podemos emplear tantas instrucciones **if** como sean necesarias en una estructura **if**. A este proceso se le denomina **Anidación**.

Sintaxis:

```
if (<condición1>) {
        …
        if (<condición2>) {
                …
        }
        …
}
else {
        …
        if (<condición3>) {
                …
        }
        …
}
```

Instrucciones de Control: Decisión (Anidación)

Ejemplo:

```
float ventas;
float descuento;
…
if (ventas > 10000) {
        descuento = 0.10 * ventas;
}
else {
        if (ventas > 5000) {
                descuento = 0.05 * ventas;
        }
        else {
                descuento = 0.02 * ventas;
        }
}
…
```

Funciones de Comprobación de Datos

Las siguientes funciones permiten realizar la conversión de una cadena de caracteres a un dato de un tipo concreto y comprobar si ésta conversión se ha realizado con éxito:

Función de Conversión / Comprobación	Conversión y Valor Devuelto
Int32.TryParse (<cadena>, out varInt)	Conversión cadena de caracteres a int
UInt32.TryParse (<cadena>, out varUInt)	Conversión cadena de caracteres a uint
Single.TryParse (<cadena>, out varFloat)	Conversión cadena de caracteres a float
Double.TryParse (<cadena>, out varDouble)	Conversión cadena de caracteres a double
Decimal.TryParse (<cadena>, out varDecimal)	Conversión cadena de caracteres a decimal

Las funciones devuelven **true** si la conversión se ha realizado con éxito y **false** si se ha producido un error en la conversión, la cadena no es un número ó sobrepasa el rango de valores del tipo de datos.

Instrucciones de Control: Decisión Múltiple

La instrucción **switch** permite a la aplicación ejecutar uno o varios bloques de código según el valor de una expresión.

Instrucción apropiada para situaciones con muchas opciones (lista de valores posibles).

Sintaxis:

```
switch (<expresión>) {
    case <valor1>:
        <InstruccionesValor1>
        break;
    case <valor2>:
        <InstruccionesValor2>
        break;
    …
    default:
        <InstruccionesDefault>
        break;
}
```

La ***<expresión>*** de control del **switch** debe ser una variable o una expresión que proporcione un valor definido dentro de los siguientes tipos:

- Enteros: **sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**
- Lógicos: **bool**
- Caracteres: **char**
- Cadenas: **string**

Instrucciones de Control: Decisión Múltiple

Ejemplo:

```
float ventas;
float descuento;
int opcion;
…
switch (opcion) {
        case 1:
                descuento = 0;
                break;
        case 2:
        case 3:
                descuento = 0.05 * ventas;
                break;
        case 4:
        case 5:
        case 6:
                descuento = 0.10 * ventas;
                break;
        default:
                descuento = 0.15 * ventas;
                break;
}
…
```

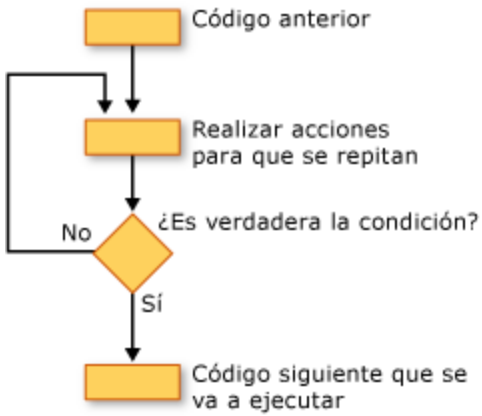
Instrucciones de Control: Iterativas (Bucles)

Las **estructuras de bucles** de **C#** permiten ejecutar una o más líneas de código de forma repetitiva.

Repiten las instrucciones de una estructura de bucles hasta que:

- Una condición sea **true**.
- Una condición sea **false**.
- Un número de veces especificado o determinado.
- *Una vez para cada objeto de una **"Colección"**.*

Ejemplo:



Instrucciones de Control: Iterativas (Bucles) WHILE

La instrucción **while** ejecuta un conjunto de instrucciones mientras la condición especificada en la instrucción **while** sea **true**.

Si la condición es **false** cuando entra en el bucle por primera vez, no se ejecuta ni una vez.

Sintaxis:

```
while (<condición>) {
    < Instrucciones >
    [ break; ]
    < Instrucciones >
}
```

Transfiere el control fuera del bloque **while**.

Instrucciones de Control: Iterativas (Bucles) WHILE

Ejemplo:

```
int contador = 0;
…
while (contador < 20) {
    Console.WriteLine (contador);
    contador = contador + 1;
}
…
```

Evaluar el resultado de la ejecución del bucle.

Instrucciones de Control: Iterativas (Bucles) DO - WHILE

La instrucción **do...while** permite probar una condición al final de una estructura de bucle.

En la condición se especifica la repetición de las instrucciones inmersas en el bucle mientras la condición sea **true**.

A diferencia de la instrucción **while**, un bucle **do-while** se ejecuta una vez antes de que se evalúe la expresión condicional.

Sintaxis:

```
do {
    < Instrucciones >
    [ break; ]
    < Instrucciones >
} while (<condición>);
```

Transfiere el control fuera del bloque **do-while**.

Instrucciones de Control: Iterativas (Bucles) DO - WHILE

Ejemplo:

```
int contador = 0;
…
do {
    Console.WriteLine (contador);
    contador = contador + 1;
} while (contador < 20);
…
```

Evaluar el resultado de la ejecución del bucle.

Instrucciones de Control: Iterativas (Bucles) FOR

La construcción **for** ejecuta el bucle un número fijo de veces.

Utiliza una variable de control de bucle, también denominada **contador**, para realizar el seguimiento de las repeticiones.

Especifica los **valor de inicio** de este **contador** y en la **condición** se evalúa si se prosigue con la repetición de las instrucciones del bucle.

Se puede especificar opcionalmente la **cantidad en la que se incrementa** el contador de una repetición a la siguiente.

Sintaxis:

For (<valorInicial>**;** <condición>**;** <incremento>**) {**

<Instrucciones>

[**break;**] ← Transfiere el control fuera del bloque **for**.

<Instrucciones>

}

Instrucciones de Control: Iterativas (Bucles) FOR

Ejemplo:

```
int contador;
…
for (contador = 0; contador < 20; contador++) {
    Console.WriteLine (contador);
}
…
```

Instrucciones de Control: Iterativas (Bucles)

Conclusiones / Comparativa:

Tipos de Datos: Conversión y Casting

C# es un lenguaje con *establecimiento inflexible de tipos*, es decir, todas las variables y constantes deben estar definidas con un tipo, al igual que toda expresión da como resultado un valor.

La ***biblioteca de clases* .NET Framework** define un conjunto de tipos numéricos integrados y tipos más complejos. Un programa típico de **C#** emplea los tipos proporcionados por la ***biblioteca de clases***.

La información almacenada en un tipo puede incluir lo siguiente:

- El **espacio de almacenamiento** que requiere una variable del tipo.
- Los **valores máximo y mínimo** que puede representar.
- La ubicación donde se asignará la memoria para las variables en tiempo de ejecución.
- Los tipos de operaciones permitidos.

El compilador emplea la información de tipos para asegurarse que las operaciones realizadas en el código cumplen con las reglas de seguridad de tipos.

Tipos de Datos: Conversión y Casting

Tabla de Tipos Integrados: Correspondencia de los tipos de datos empleados en **C#** con los proporcionados por la ***biblioteca de clases*** de **.NET Framework**.

Tipo C#	Tipo .NET Framework	Tipo C#	Tipo .NET Framework
bool	System.Boolean	uint	System.UInt32
byte	System.Byte	long	System.Int64
sbyte	System.SByte	ulong	System.UInt64
char	System.Char	object	System.Object
decimal	System.Decimal	short	System.Int16
double	System.Double	ushort	System.UInt16
float	System.Single	string	System.String
int	System.Int32		

Todos los tipos de la tabla, excepto **object** y **string**, se conocen como **Tipos Simples**.

Tipos de Datos: Conversión y Casting

Durante la ejecución del código del programa, en ocasiones, podemos necesitar que las variables declaradas de un tipo de datos contengan valores de otros tipos de datos.

Estos tipos de operaciones se denominan Conversiones de Tipos. En C#, puede realizar los siguientes tipos de conversiones:

- **Conversiones Implícitas**: No se requiere una sintaxis especial porque la conversión se realiza con tipos equivalentes y no se perderán datos. Ejemplo: Tipos enteros de menor a mayor.

De	A	De	A
sbyte	short, int, long, float double ó decimal	uint	u/long, float, double ó decimal
byte	u/short, u/int, u/long, float, double ó decimal	long	float, double ó decimal
short	int, long, float, double ó decimal	char	ushort, u/int, u/long, float, double ó decimal
ushort	u/int, u/long, float, double ó decimal	float	double
int	long, float, double ó decimal	ulong	float, double ó decimal

Tipos de Datos: Conversión y Casting

- **Conversiones Explícitas**: Las conversiones explícitas requieren un operador de conversión. A este tipo de conversiones se les suele denominar también como **Conversiones de Tipo** (**Casting**).

Estas operaciones informan a al compilador de forma explícita que se pretende realizar una conversión y que está al tanto de que puede producirse una pérdida de datos (el tipo de la variable de destino es más pequeño que la variable de origen).

Para realizar una conversión de tipo, se especifica entre paréntesis el tipo al que se va a aplicar dicha conversión delante del valor o la variable que se va a convertir.

Ejemplo:

```
double y = 3456.7;
int x;
x = (int)y;      // x contiene 3456
```

Tipos de Datos: Conversión y Casting

Tabla de Conversiones Numéricas Explícitas

De	A	De	A
sbyte	byte, ushort, uint, ulong, ó char	long	sbyte, byte, short, ushort, int, uint, ulong ó char
byte	sbyte ó char	ulong	sbyte, byte, short, ushort, int, uint, long ó char
short	sbyte, byte, ushort, uint, ulong ó char	char	sbyte, byte ó short
ushort	sbyte, byte, short ó char	float	sbyte, byte, short, ushort, int, uint, long, ulong, char ó decimal
int	sbyte, byte, short, ushort, uint, ulong ó char	double	sbyte, byte, short, ushort, int, uint, long, ulong, char, float ó decimal
uint	sbyte, byte, short, ushort, int ó char	decimal	sbyte, byte, short, ushort, int, uint, long, ulong, char, float ó decimal

Tipos de Datos: Conversión y Casting

Conversiones Numéricas Explícitas: A tener en cuenta

- Las conversiones numéricas explícitas pueden producir ***pérdida de precisión*** o ***provocar excepciones***.

- Cuando se convierte un valor **double** o **float** en un tipo **entero**, el valor se trunca. Si el valor entero resultante queda fuera del rango de valores establecido para el tipo de destino, se produce una excepción **OverflowException**.

- Cuando se convierte **double** en **float**, el valor **double** se redondea al valor **float** más próximo. Si el valor de tipo **double** es demasiado pequeño o demasiado grande para ajustarse al tipo de destino, el resultado será **cero** o **infinito**.

Tipos de Datos: Conversión y Casting

- **Conversiones Explícitas**: Para realizar conversiones entre tipos no compatibles, como los enteros y las cadenas de caracteres (**string**) se puede emplear:

  - La clase **System.Convert**:

    Convert.ToInt32(cadena)

    Convert.ToDouble(cadena)

    Etc.

  - Los métodos **Parse** y **TryParse** de los tipos numéricos:

    Int32.TryParse (cadena, out entero)

    entero = **Int32.Parse** (cadena)

    Etc.

Inicialización de Variables

El compilador de C# es muy estricto a la hora de detectar si las variables que se han declarado han sido utilizadas a lo largo del código del programa.

En el caso de que a una variable se le asigne un valor dentro de una **Sentencia de Control de Decisión**:

Ejemplo:

```
byte variable;
…
if (<condición>) {
        variable = <valor>;
}
```

Y posteriormente se quiera hacer uso del valor almacenado en dicha variable, el compilador detectará un error y no permitirá la ejecución del programa.

Para evitar estos errores es conveniente **Inicializar las Variables** declaradas con un valor.

4.- Operadores Especiales (C#)

Operadores de Incremento y Decremento

Operador de Incremento ++

Realiza el incremento del valor del operando en 1.

El operador de incremento puede aparecer antes o después de su operando:

- **Pre-Incremento**: El resultado de la operación es el valor del operando después de haber sido incrementado.

Ejemplo:

```
int x = 55;
Console.WriteLine (++x); // Muestra por pantalla 56
```

- **Post-Incremento**: El resultado de la operación es el valor del operando antes de haber sido incrementado.

Ejemplo:

```
int x = 55;
Console.WriteLine (x++); // Muestra por pantalla 55
Console.WriteLine (x);   // Muestra por pantalla 56
```

Operadores de Incremento y Decremento

Operador de Decremento --

Realiza el decremento del valor del operando en 1.

El operador de decremento puede aparecer antes o después de su operando:

- **Pre-Decremento**: El resultado de la operación es el valor del operando después de haber sido decrementado.

Ejemplo:

```
int x = 55;
Console.WriteLine (--x);  // Muestra por pantalla 54
```

- **Post-Decremento**: El resultado de la operación es el valor del operando antes de haber sido decrementado.

Ejemplo:

```
int x = 55;
Console.WriteLine (x--);  // Muestra por pantalla 55
Console.WriteLine (x);    // Muestra por pantalla 54
```

4.- Operadores Especiales (C#)

Operadores de Asignación

En programación es habitual encontrarnos con operaciones del tipo siguiente:

x = x + <valor>

Donde **x** es una variable de tipo numérico y **<valor>** es un literal que indica la cantidad con la que se quiere incrementar la variable (en este caso).

C# provee unos **Operadores de Asignación** especiales que permiten simplificar estas expresiones:

+=

-=

***=**

/=

4.- Operadores Especiales (C#)

Operadores de Asignación

Operador de Asignación y Suma +=

x **+=** y ----- x = x + y

Operador de Asignación y Resta -=

x **-=** y ----- x = x - y

Operador de Asignación y Producto *=

x ***=** y ----- x = x * y

Operador de Asignación y División /=

x **/=** y ----- x = x / y

4.- Operadores Especiales (C#)

Operadores Lógicos

Operador &

- Este operador empleado para evaluar expresiones lógicas, calcula la operación lógica **AND** (*Conjunción Lógica*) de sus operandos; es decir, el resultado es **true** si, y sólo si ambos operandos son **true**.

  <expresión1> **&** <expresión2>

- El operador **&** evalúa ambos operadores sin tener en cuenta el valor del primero.

Operador &&

- Este operador realiza una operación lógica **AND** (*Conjunción Lógica*) de sus operandos de tipo bool, pero sólo evalúa el segundo operando si es necesario.

  <expresión1> **&&** <expresión2>

- En el caso de que *<expresión1>* fuese **false**, *<expresión2>* no se evaluaría. Esto se conoce como **evaluación "cortocircuitada"**.

4.- Operadores Especiales (C#)

Operadores Lógicos

Operador |

- Este operador empleado para evaluar expresiones lógicas, calcula la operación lógica **OR** (*Disyunción Lógica*) de sus operandos; es decir, el resultado es **true** si uno de los operandos es **true**.

  <expresión1> **|** <expresión2>

- El operador | evalúa ambos operadores sin tener en cuenta el valor del primero.

Operador ||

- Este operador realiza una operación lógica **OR** (*Disyunción Lógica*) de sus operandos de tipo bool, pero sólo evalúa el segundo operando si es necesario.

  <expresión1> **||** <expresión2>

- En el caso de que <*expresión1*> fuese **true**, <*expresión2*> no se evaluaría. Esto se conoce como **evaluación "cortocircuitada"**.